Front End Design Document

Digital Games Distribution System – Phase 2

CSCI 3060U, Fall 2026 – Team Sabre Inc.

Chris Ibrahim, Faraz Shaikh, Jesse Parker, Mir Mahdi Ali

1. General Overview

The following is the front-end design documentation for Phase 2 of the system. Section 3 contains the UML Class Diagram and Section 4 includes a brief description of each class and method. The classes were determined based on each behavior described in the 16 Test Cases where we asked ourselves, "which class will handle each behavior" as we went through the test cases individually. Section 5 maps each test case to the code that handles it.

2. Design Decisions

Each command was given its own Transaction Subclass (login, logout, create, delete, sell, buy, refund, addcredit, list, listaccounts). A TransactionFactory determines which Transaction subclass should be used to process the user's command and if it cannot determine which to use it simply returns nothing thus causing no issues when processing incorrect or unknown commands as this will simply print an error message and continue running (Test Case 16).

All of the checks that many commands share such as "Is anyone logged in?", "Is this an Admin?", etc. are placed within the Transaction Base Class so that these checks need to be written only once (Tests 2 & 3).

The three input files; Current User Accounts File, Available Games File, and Game Collection File, are each loaded into separate objects (AccountFile, GameFile, and CollectionFile). Once a new game is listed for sale, it will be placed into a Pending List in GameFile and will become available for purchase once the user logs out of their account (Test Case 9).

Session maintains information about the current user logged in including how much credit they've added so far in the session which allows us to track whether they have exceeded the total allowable amount of $1000.00 using multiple addcredit commands (Test Case 11).

Only the DailyTransactionWriter is allowed to make changes to the Daily Transaction File. Commands that fail and Read-Only List commands do not call DailyTransactionWriter therefore the Daily Transaction File only includes successful transactions (Test Cases 13-15).

The input validation rules (maximum username size = 15 characters, maximum game name size = 25 characters, maximum price = $999.99 and only numbers are allowed) for all possible commands are located in InputValidator so there is no duplication of code across all Transaction subclasses.

3. UML class diagram

Figure 1 covers how commands get handled. Figure 2 covers the data those commands work with. FrontEndContext is in both because it is the link between them.

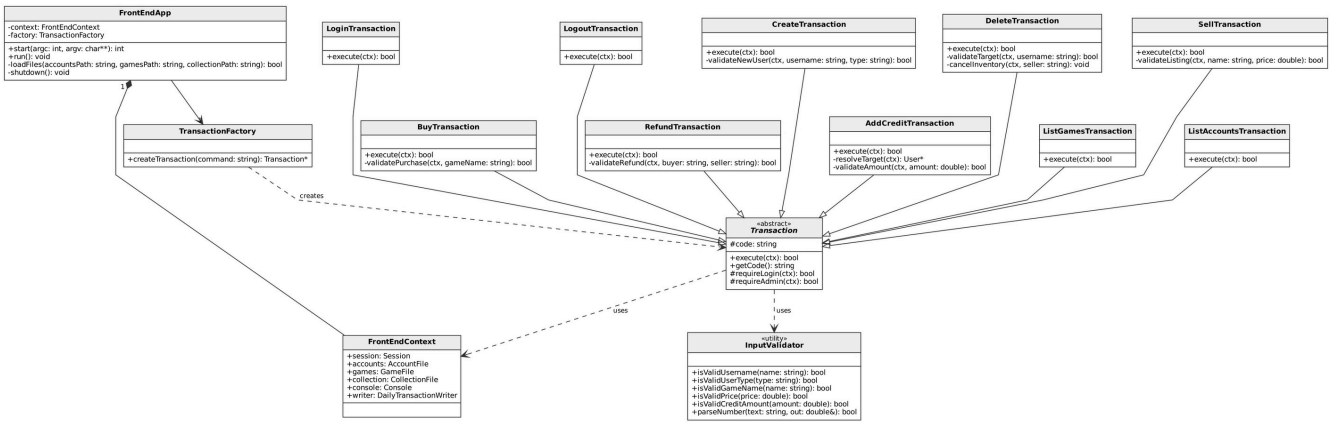


Figure 1. Command handling: FrontEndApp, TransactionFactory, and the Transaction classes.

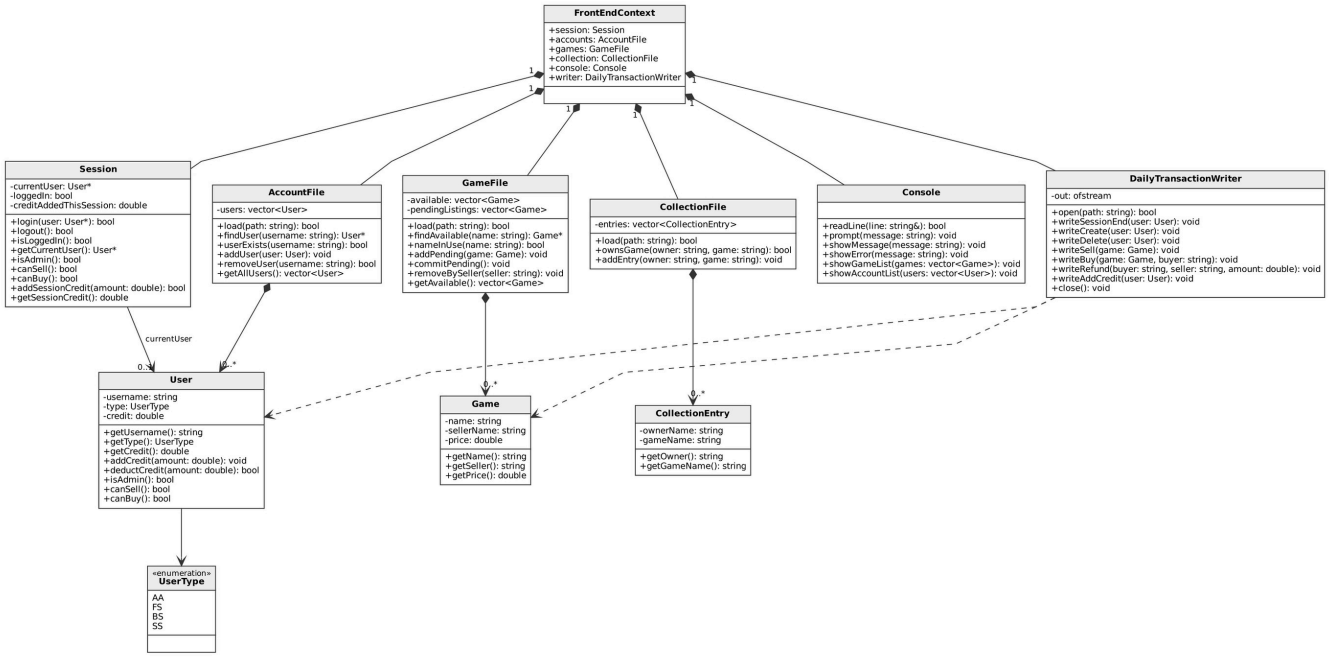


Figure 2. State and data: FrontEndContext, Session, the in-memory files, and their records.

4. Class and method descriptions

+ means public, - private, and # protected. In the Transaction subclasses, ctx is the shared FrontEndContext that gets passed to every execute call.

Class / method	Description (intent)
FrontEndApp (class)	Starts the program, loads the three input files, reads commands from standard input, and saves the Daily Transaction File at the end.
+start(argc: int, argv: char**): int	Validates the four file-path arguments and starts the application.
+run(): void	Reads commands until end of input and executes the matching transaction for each one.
-loadFiles(accountsPath: string, gamesPath: string, collectionPath: string): bool	Loads the three input files into memory and reports whether all of them loaded.
-shutdown(): void	Flushes and closes the Daily Transaction File once input ends.

Class / method	Description (intent)
FrontEndContext (class)	Holds the shared objects (session, files, console, writer) that every transaction needs.
Session (class)	Keeps track of who is logged in and how much credit they have added this session.
+login(user: User*): bool	Starts a session for the given user unless a session is already active.
+logout(): bool	Ends the active session and clears its state, failing if no one is logged in.
+isLoggedIn(): bool	Reports whether a session is currently active.
+getCurrentUser(): User*	Returns the user who is currently logged in.
+isAdmin(): bool	Reports whether the logged-in user is an administrator.
+canSell(): bool	Reports whether the logged-in user is allowed to sell games.
+canBuy(): bool	Reports whether the logged-in user is allowed to buy games.
+addSessionCredit(amount: double): bool	Adds to the session credit total only if the $1000.00 session limit would not be exceeded.
+getSessionCredit(): double	Returns the total credit added so far in this session.
User (class)	One account record holding a username, account type, and available credit.
+getUsername(): string	Returns the account's username.
+getType(): UserType	Returns the account's type code.
+getCredit(): double	Returns the account's available credit.

Class / method	Description (intent)
+addCredit(amount: double): void	Increases the account's available credit by the given amount.
+deductCredit(amount: double): bool	Decreases the available credit if enough is available and reports success.
+isAdmin(): bool	Reports whether the account type is administrator (AA).
+canSell(): bool	Reports whether the account type may sell games (AA, FS, SS).
+canBuy(): bool	Reports whether the account type may buy games (AA, FS, BS).
UserType (enumeration)	The four legal account types: admin (AA), full-standard (FS), buy-standard (BS), and sell-standard (SS).
Game (class)	One game listing with its name, seller, and price.
+getName(): string	Returns the game's name.
+getSeller(): string	Returns the username of the game's seller.
+getPrice(): double	Returns the game's price.
CollectionEntry (class)	One record stating that a given user owns a given game.
+getOwner(): string	Returns the username of the owner.
+getGameName(): string	Returns the name of the owned game.
AccountFile (class)	In-memory copy of the Current User Accounts File with lookup, add, and remove operations.
+load(path: string): bool	Reads the Current User Accounts File and reports whether it loaded.
+findUser(username: string): User*	Returns the matching account or null if it does not exist.

Class / method	Description (intent)
+userExists(username: string): bool	Reports whether an account with that username exists.
+addUser(user: User): void	Adds a newly created account to the in-memory list.
+removeUser(username: string): bool	Removes the named account and reports whether it was found.
+getAllUsers(): vector<User>	Returns every active account for listing.
GameFile (class)	In-memory copy of the Available Games File; new listings wait here until the next session.
+load(path: string): bool	Reads the Available Games File and reports whether it loaded.
+findAvailable(name: string): Game*	Returns the matching game if it is available for purchase this session, else null.
+nameInUse(name: string): bool	Reports whether a game name is already listed, counting pending listings.
+addPending(game: Game): void	Records a newly listed game that cannot be bought until the next session.
+commitPending(): void	Moves pending listings into the available list when a session ends.
+removeBySeller(seller: string): void	Cancels every available and pending listing owned by the given seller.
+getAvailable(): vector<Game>	Returns all games currently available for purchase.
CollectionFile (class)	In-memory copy of the Game Collection File used to check what each user already owns.
+load(path: string): bool	Reads the Game Collection File and reports whether it loaded.
+ownsGame(owner: string, game: string): bool	Reports whether the user already owns the game.

Class / method	Description (intent)
+addEntry(owner: string, game: string): void	Records a purchased game in the buyer's collection.
Console (class)	Takes care of everything that is read from or printed to the terminal.
+readLine(line: string&): bool	Reads one line from standard input and returns false at end of input.
+prompt(message: string): void	Prints a prompt asking the user for the next piece of input.
+showMessage(message: string): void	Prints a success or informational message.
+showError(message: string): void	Prints an error message for a rejected command.
+showGameList(games: vector<Game>): void	Prints every available game with its seller and price, or says none are available.
+showAccountList(users: vector<User>): void	Prints every active account with its username, type, and available credit.
DailyTransactionWriter (class)	Formats each transaction record and writes it to the Daily Transaction File in the order things happen.
+open(path: string): bool	Opens the Daily Transaction File for writing.
+writeSessionEnd(user: User): void	Writes the 00 end-of-session record for the logged-out user.
+writeCreate(user: User): void	Writes the 01 record for a newly created account.
+writeDelete(user: User): void	Writes the 02 record for a deleted account.
+writeSell(game: Game): void	Writes the 03 record for a new game listing.
+writeBuy(game: Game, buyer: string): void	Writes the 04 record with the game, seller, buyer, and price.

Class / method	Description (intent)
+writeRefund(buyer: string, seller: string, amount: double): void	Writes the 05 record for a refund between a buyer and a seller.
+writeAddCredit(user: User): void	Writes the 06 record for a credit addition.
+close(): void	Closes the Daily Transaction File.
InputValidator (utility class)	Helper class with the checks for field lengths, valid types, and number ranges.
+isValidUsername(name: string): bool	Checks that a username is not longer than 15 characters.
+isValidUserType(type: string): bool	Checks that an account type is one of AA, FS, BS, or SS.
+isValidGameName(name: string): bool	Checks that a game name is not longer than 25 characters.
+isValidPrice(price: double): bool	Checks that a price is not negative and not above $999.99.
+isValidCreditAmount(amount: double): bool	Checks that a single credit addition is positive and not above $1000.00.
+parseNumber(text: string, out: double&): bool	Converts text to a number and rejects nonnumeric input.
TransactionFactory (class)	Picks the right Transaction object for whatever command the user typed.
+createTransaction(command: string): Transaction*	Returns the matching transaction, or null if the command isn't recognized.
Transaction (abstract class)	Base class for every command; it defines execute() and the login and admin checks the commands share.
+execute(ctx: FrontEndContext&): bool	Runs the command and returns true only if it succeeded.
+getCode(): string	Returns the two-digit transaction code, or empty for unlogged commands.

Class / method	Description (intent)
#requireLogin(ctx: FrontEndContext&): bool	Prints an error and returns false if no user is logged in.
#requireAdmin(ctx: FrontEndContext&): bool	Prints an error and returns false unless the logged-in user is an administrator.
LoginTransaction (class)	Handles the login command (no record is written).
+execute(ctx: FrontEndContext&): bool	Prompts for a username and starts a session if the account exists and no session is already active.
LogoutTransaction (class)	Handles the logout command (transaction code 00).
+execute(ctx: FrontEndContext&): bool	Ends the session, commits pending game listings, and writes the 00 record, rejecting the command if no one is logged in.
CreateTransaction (class)	Handles the administrator-only create command (transaction code 01).
+execute(ctx: FrontEndContext&): bool	Prompts for a username and type, creates the account, and writes the 01 record.
-validateNewUser(ctx: FrontEndContext&, username: string, type: string): bool	Rejects duplicate usernames, usernames over 15 characters, and invalid account types.
DeleteTransaction (class)	Handles the administrator-only delete command (transaction code 02).
+execute(ctx: FrontEndContext&): bool	Prompts for a username, deletes the account, and writes the 02 record.
-validateTarget(ctx: FrontEndContext&, username: string): bool	Rejects nonexistent users and the user who is currently logged in.
-cancelInventory(ctx: FrontEndContext&, seller: string): void	Removes the deleted seller's games so nobody can buy them afterwards.
SellTransaction (class)	Handles the sell command (transaction code 03).

Class / method	Description (intent)
+execute(ctx: FrontEndContext&): bool	Prompts for a game name and price, lists the game as pending, and writes the 03 record.
-validateListing(ctx: FrontEndContext&, name: string, price: double): bool	Rejects buy-standard sellers, long names, prices over $999.99, and duplicate game names.
BuyTransaction (class)	Handles the buy command (transaction code 04).
+execute(ctx: FrontEndContext&): bool	Prompts for a game, transfers the credit, adds the game to the buyer's collection, and writes the 04 record.
-validatePurchase(ctx: FrontEndContext&, gameName: string): bool	Rejects sell-standard buyers, unavailable games, insufficient credit, and games already owned.
RefundTransaction (class)	Handles the administrator-only refund command (transaction code 05).
+execute(ctx: FrontEndContext&): bool	Prompts for buyer, seller, and amount, moves the credit back to the buyer, and writes the 05 record.
-validateRefund(ctx: FrontEndContext&, buyer: string, seller: string): bool	Rejects a refund when the buyer or the seller does not exist.
AddCreditTransaction (class)	Handles the addcredit command (transaction code 06).
+execute(ctx: FrontEndContext&): bool	Prompts for an amount, applies it to the target account, and writes the 06 record.
-resolveTarget(ctx: FrontEndContext&): User*	Returns the logged-in user for standard accounts, or prompts for and looks up the target username for administrators.
-validateAmount(ctx: FrontEndContext&, amount: double): bool	Rejects a single amount over $1000.00 or one that would exceed the session limit.
ListGamesTransaction (class)	Handles the read-only list command that shows available games (unlogged).

Class / method	Description (intent)
+execute(ctx: FrontEndContext&): bool	Prints every available game with its seller and price, or reports that none are available.
ListAccountsTransaction (class)	Handles the administrator-only, read-only listaccounts command (unlogged, assumed keyword).
+execute(ctx: FrontEndContext&): bool	Prints every active account with its username, type, and available credit.

5. Test-case traceability

The classes and methods that each test in test_cases/ relies on.

Test	Scenario	Classes and methods that satisfy it
1	Valid login/logout	LoginTransaction.execute, LogoutTransaction.execute, Session.login/logout, DailyTransactionWriter.writeSessionEnd
2	Invalid session operations	Transaction.requireLogin, Session.isLoggedIn, Session.login, LoginTransaction.execute, LogoutTransaction.execute
3	Account privilege restrictions	Transaction.requireAdmin, Session.isAdmin, Create/Delete/Refund/ListAccountsTransaction.execute
4	Create valid user types	CreateTransaction.execute, AccountFile.addUser, InputValidator.isValidUserType, DailyTransactionWriter.writeCreate
5	Create invalid inputs	CreateTransaction.validateNewUser, AccountFile.userExists, InputValidator.isValidUsername/isValidUserType
6	Delete user behaviours	DeleteTransaction.execute/validateTarget/cancelInventory, AccountFile.removeUser, GameFile.removeBySeller, DailyTransactionWriter.writeDelete

Test	Scenario	Classes and methods that satisfy it
7	Sell behaviours	SellTransaction.execute/validateListing, Session.canSell, GameFile.nameInUse/addPending, InputValidator.isValidGameName/isValidPrice, DailyTransactionWriter.writeSell
8	Buy behaviours	BuyTransaction.execute/validatePurchase, Session.canBuy, GameFile.findAvailable, CollectionFile.ownsGame/addEntry, User.deductCredit, DailyTransactionWriter.writeBuy
9	Newly listed game, next session	GameFile.addPending/commitPending, LogoutTransaction.execute, BuyTransaction.validatePurchase
10	Refund behaviours	RefundTransaction.execute/validateRefund, AccountFile.findUser, User.addCredit/deductCredit, DailyTransactionWriter.writeRefund
11	Standard addcredit	AddCreditTransaction.execute/validateAmount, Session.addSessionCredit, InputValidator.isValidCreditAmount, DailyTransactionWriter.writeAddCredit
12	Admin addcredit	AddCreditTransaction.resolveTarget, AccountFile.findUser, Session.isAdmin, DailyTransactionWriter.writeAddCredit
13	List available games	ListGamesTransaction.execute, GameFile.getAvailable, Console.showGameList
14	List with no available games	ListGamesTransaction.execute, Console.showGameList (empty case)
15	List active accounts	ListAccountsTransaction.execute, AccountFile.getAllUsers, Console.showAccountList
16	Malformed input recovery	TransactionFactory.createTransaction (null on unknown), InputValidator.parseNumber, Console.readLine/showError, FrontEndApp.run

6. Assumptions

- Although we're not specifying the exact format for each record in the Daily Transaction File (see the handout), the DailyTransactionWriter will follow it and we can use the records from the test suite to demonstrate how they are supposed to be formatted.
- There is no "admin accounts" write transaction. The admin-only command to view all active accounts is called listaccounts. Like list, this only reads the data and does NOT create a record. If the professor or TA calls it something else or wants it to do something differently than list, we'll need to modify ListAccountsTransaction.
- When a game is listed for sale during a session, it becomes available to buy once the seller logs off at the end of the session.
- We run the program as frontend <accounts_file> <games_file> <collection_file> <daily_output_file>, with commands coming in on standard input.